Bevezető

Az AngularJS-t érdemes úgy kézbe venni, mintha egy okos réteget húzna a hagyományos HTML fölé: ugyanazokat a címkéket és űrlapokat használja, mégis „életre kelnek”. Az oldal nem pusztán statikus jelölőnyelv többé, hanem adatok nézete. Az interpolációval a sablon konkrét értékeket jelenít meg; a kétirányú adatkötés pedig biztosítja, hogy amit a felhasználó beír egy mezőbe, az azonnal megjelenjen a nézetben, és fordítva: ha a program állítja át a modellt, a felület rögtön követi. Így megszűnik az állandó kézi DOM-kezelés kényszere: nem „hozzáír a HTML-hez”, hanem megmondja, hogy a felület mit „tükrözzön” az adatokból és a keretrendszer elvégzi a köztes munkát. Már az első pillanattól érdemes következetesen bánni az üres értékekkel is: ha a felhasználó még nem írt nevet, ne üres helyet lásson, hanem kulturált helyettesítő szöveget; ettől lesz a felület koherens és barátságos.

A keretrendszer szerkezetének megértése a következő lépés. Az alkalmazás egy modulba kerül; ez a névtér, ahová a részeket – vezérlőket, szolgáltatásokat, komponenseket – regisztrálja. A nézet és a logika kapcsolata a vezérlőn keresztül történik. Ajánlott a „ControllerAs” minta: a vezérlő „this” értékére helyezi az adatokat és a műveleteket, és a nézetben egy rövid, beszédes aliason éri el őket. Ennek egyszerű oka van: olvashatóbb, könnyebb tesztelni és nem keveredik össze több különböző forrásból származó változó. A nézet így mindig egyetlen, tiszta névtérre hivatkozik, ami nemcsak esztétikusabb, hanem a hibakeresést is megkönnyíti.

Amint megvan a váz, életet kell vinni a felületbe: a gomboknak és űrlapoknak reakciót kell kiváltaniuk. **Az eseménykezelés** AngularJS-ben egyszerű: a nézet meghívja a vezérlőben megírt műveleteket és azok módosítják a modellt. Tipikus helyzet egy beviteli mező és egy lista: a felhasználó beír valamit, beküldi az űrlapot, a vezérlő ellenőrzi, hogy nem üres-e a bevitel, hozzáadja a tömbhöz, majd kiüríti a mezőt. A jó élményhez az apró részletek számítanak: felesleges üres elemeket ne engedjen be a rendszer és legyen kényelmes mód a lista ürítésére is. A felület visszajelzései – például egy rövid „mentve” üzenet – különösen az első lépések során sokat segítenek.

A listák megjelenítése és szűrése az AngularJS egyik legkényelmesebb területe. Gondolkodjon úgy, hogy van egy tömbje, és a sablon mindegyik elemre ugyanazt a minivázat ismétli. Ha keresőt szeretne, a beírt kifejezést a nézet már a megjelenítés előtt felhasználja a szűkítéshez; ez nem egy külön „újrarajzolás”, hanem ugyanennek a deklaratív szemléletnek a természetes része. A felhasználó közben nem marad bizonytalanságban: ha a keresés nem talál semmit, azt kulturáltan jelezze. Egy apró, de fontos részlet: ha objektumokat jelenít meg, ne az egész objektumon szűrjön, hanem azon a mezőn, amely a felhasználónak releváns – ettől lesz pontos és kiszámítható a találat.

A kiválasztólisták világa egy fokkal összetettebb és itt mutatkozik meg igazán az AngularJS adatmodelljének ereje. Nemcsak azt tudja megmondani, hogy a felhasználó mit lát címkeként, hanem azt is, hogy a kiválasztás eredménye mi legyen a program szempontjából. A legegyszerűbb és legtisztább, ha nem egyetlen sztringet, hanem egy

teljes objektumot tárol modellként. Egy „jegytípus” például ne csak a nevét hordozza, hanem a hozzá tartozó szorzót és magyarázó feliratot is. Így amikor a felhasználó választ, nem kell a háttérben táblázatokat keresgélnie: a szükséges adatok már a modellben vannak. Egy dologra ügyeljen: ha alapértelmezett választást szeretne, ténylegesen azt az objektumot adja a modellnek, amely a listában is szerepel, ne egy vele azonos tartalmú, de újonnan létrehozott példányt – az AngularJS a valódi azonosságot figyeli, nem pusztán az „ugyanolyan értékeket”.

A számítás – különösen űrlapadatokkal – a gyakorlatban mindig előhozza a típusok kérdését. A mezőkből gyakran szövegek érkeznek, miközben Ön számokkal szeretne dolgozni. Ilyenkor legyen egy tudatos, egységes belépési pont: először alakítsa számmá a bevitelt, hibás vagy üres érték esetén állítsa alaphelyzetbe nullára, majd kényszerítse a 0 alá eső tartományt minimum nullára. E három egyszerű lépésből stabil logika születik, amelyre bátran építhet: a nettó összeg nem lesz negatív, az adó a helyes alapra számítódik, és a végösszeg nem „fut szét” attól, hogy valaki félregépelt. Az árkalkulátor klasszikus sorrendje – nettó összesen, kedvezmény levonása, adó a kedvezmény utáni összegre, bruttó végösszeg – ebben a formában nemcsak kézenfekvő, hanem pénzügyi szempontból is helyes.

Az űrlapoknál érdemes kihasználni az AngularJS saját **állapotjelzés**eit. Egy mező lehet még érintetlen, lehet módosított, és lehet érvénytelen vagy érvényes. Ezek együtt adják a jó felhasználói élményt: a hibákat nem szükséges azonnal pirosan megjeleníteni; elég akkor, amikor a felhasználó már interakcióba lépett a mezővel. A beküldés gomb állapota tükrözheti az űrlap egészének érvényességét. Ez a megközelítés nem csupán udvariasabb, hanem ténylegesen csökkenti a hibázást és gyorsítja a kitöltést.

A dinamikus stílus végül megmutatja, hogy az adatok nemcsak szövegeket, hanem megjelenést is meghatározhatnak. Ha a felhasználó kiválaszt egy színt, a felület egy kis előnézeti dobozban rögtön képes azt megjeleníteni. Ha állapotokat kezel, a sorok – például – zöldek, sárgák vagy pirosak lehetnek az állapot szerint. A lényeg itt is változatlan: az adatok diktálnak, a nézet pedig tükröz. Érdemes alapértelmezett értéket adni, hogy a felület betöltés után azonnal koherens legyen, és gondoskodni arról, hogy a vizuális elemek (keret, méret) a lehető legvilágosabban jelezzék a különbségeket.

Gyors start – eszközök

- **VS Code** + **Live Server** bővítmény (jobb klikk → *Open with Live Server*).
- **CDN-ek**:
    - AngularJS: https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js
    - Bootstrap 5: CSS a gyors, szép stílushoz.

1) Mi az AngularJS? (és miben segít)

- **SPA keretrendszer**: egy oldalon belül váltunk nézeteket.
- **Deklaratív sablonok**: HTML-hez “okos” attribútumokat (direktívák) adunk.
- **Kétirányú adatkötés**: input és kiírás automatikusan szinkronban.
- **Függőséginjektálás (DI)**: beépített és saját szolgáltatásokat könnyen becsatlakoztatunk.
- **Szerkezet**: modulok, kontrollerek, (újabb 1.5+ verzióban) komponensek, servicék, filterek, router.

2) Interpoláció és kétirányú adatkötés

- **Interpoláció**: {{ kifejezés }} – kiírja az értéket.
- **Kétirányú kötés**: ng-model="valtozo" – input ↔ változó.

3) Direktívák – a HTML “szuperképességei”

Gyakoriak:

- ng-app (alkalmazás indítása)
- ng-model (adatkötés)
- ng-bind (kiírás, mint {{ }})
- ng-repeat (listák)
- ng-if, ng-show/ng-hide (feltételek)
- ng-click, ng-submit (események)

4) Modulok és kontrollerek (ControllerAs minta)

- **Modul**: alkalmazás “konténere”.
- **Controller**: nézethez kötött logika és adatok.
- **ControllerAs**: ng-controller="Main as vm" → a sablonban vm.*.

5) Komponensek (AngularJS 1.5+)

- A **.component()** API a modern Angular komponens gondolkodást hozza 1.x-be.
- Előny: izolált scope, egyirányú adatfolyam, könnyebb újrafelhasználás.

6) Filterek (formázás a nézetben)

- Beépítettek: number, currency, date, uppercase, lowercase, json, filter.
- Saját filter is írható.

7) Események és metódushívások

- ng-click="vm.add()", ng-submit="vm.save()", ng-change, ng-keyup, stb.

8) Űrlapok és validáció

- AngularJS saját validációs állapotokat ad: $valid, $invalid, $touched, $dirty.
- Hibajelzés Bootstrap-pel.

9) Szervizek és DI (függőséginjektálás)

- **Service / Factory**: megosztott logika, állapot, API-hívások.
- DI: a vezérlőkbe **név alapján** kapjuk meg (minifikálásnál annotáció szükséges).

10) HTTP és ígéret (Promise)

- Beépített $http szerviz (GET/POST…), ígéretet ad vissza.
- Sablonban jelöljön “Betöltés…” állapotot.

11) Routing (ngRoute)

- Több nézet egy lapon, URL alapján.
- Kell az angular-route.min.js és a modul: ['ngRoute'].

12) ng-options – profi <select>

- Minta: ng-options="t as t.name for t in vm.tickets"
    - Címke: t.name
    - Modellérték: **teljes t objektum**
- Ha csak az id kell a modellnek: t.id as t.name for t in vm.tickets

13) Számítások nézetben – jó gyakorlat

- Egyszerű, **olcsó** számítás mehet sablonba ({{ vm.net() | number:0 }}).
- Komplex logika inkább controllerben (pl. több lépéses árképzés).

14) Bootstrap – gyors, egységes UI

- Használja a container, row, col-*, card, btn, form-control, list-group osztályokat.
- Nem kell build, elég a CSS CDN.

15) Hibakeresés & tipikus buktatók

Hogyan debuggoljon?

- Böngésző DevTools (F12), *Console* hibák, *Sources* töréspontok.
- Nézetben “debug panel”: <pre>{{ vm | json }}</pre>.

Gyakori hibák

- ng-controller neve ≠ regisztrált controller → “Controller 'X'…” hiba.
- controllerAs alias keverése (hol this, hol $scope).
- ng-options-nál **referencia-egyezés**: előre kiválasztáshoz a tömb elemét adja, ne új objektumot.
- Típusok: inputból string jön → használja az unáris +-t/konverziót.
- Túl sok logika a sablonban → vigye át controllerbe.